US Accidents (2016-2023): Comprehensive Statistical Analysis Report

---

Data Analyst/BI portfolio project. This document combines the full project methodology, data dictionary, complete analysis notebook (code, narrative, and real computed results), findings, and the reasoning behind every major project decision — everything needed to evaluate both the results and the process behind them. For a shorter, non-technical version, see `executive_summary.md` / `executive_summary.pdf`.

Table of contents

Project overview

Statistical analysis of ~7.7 million US traffic accident records (2016-2023), examining what factors are associated with accident severity — weather, time of day, and road features — using hypothesis testing and logistic regression, presented alongside an interactive dashboard. Built end-to-end: data pipeline, statistical analysis, dashboard, automated tests, CI/CD, and live deployment.

Methodology

Data acquisition

Downloaded manually from the Kaggle dataset page (no Kaggle API/credentials required) into `data/raw/US_Accidents_March23.csv` (~3.06GB, ~7.7M rows). See the Project Decisions section below for why manual download was chosen over the Kaggle API.

Processing pipeline

1. `us_accidents.ingest.load_raw` registers the CSV as a DuckDB view without loading it into memory.
2. `us_accidents.ingest.validate_schema` checks the columns in the Data Dictionary (below) are present.
3. `us_accidents.aggregate` produces:
4. Group-by rollups (by state, weather, hour, year) for the dashboard and descriptive statistics.
5. A 200,000-row reproducible random sample (seed 42) for hypothesis testing and regression — full-dataset granularity isn't needed for valid inference at this scale.

6. All outputs are written to `data/processed/` as Parquet (plus a JSON summary of the regression results) and committed to git — small, derived files, unlike the raw CSV.

Run via: `uv run python scripts/build_aggregates.py`

Development environment note

The project lives on an external exFAT-formatted drive, which doesn't support filesystem hardlinks. `uv sync` therefore does full file copies instead of the usual instant hardlinks — installs/reinstalls are slower than on an NTFS drive, but functionally unaffected. Accepted as a known tradeoff rather than relocating the project.

Data dictionary

Source: US Accidents (2016-2023) by Sobhan Moosavi. Columns below are the ones this analysis actually uses (the raw file has 46 columns total).

Column	Type	Meaning
Severity	int (1-4)	Impact on traffic, 1 = least, 4 = most severe
Start_Time	timestamp	When the accident began
State	string	US state abbreviation
Weather_Condition	string	Weather at the time of the accident
Temperature(F)	float	Temperature in Fahrenheit
Visibility(mi)	float	Visibility in miles
Junction	bool	Whether the accident occurred near a road junction
Crossing	bool	Whether the accident occurred near a crossing
Traffic_Signal	bool	Whether a traffic signal was present nearby
Stop	bool	Whether a stop sign was present nearby
Sunrise_Sunset	string	"Day" or "Night" at the time of the accident

---

Full analysis

The following is the complete, executed analysis notebook (`notebooks/01_eda_and_hypothesis_testing.ipynb`) — all code, narrative, and real computed results reproduced directly, not summarized.

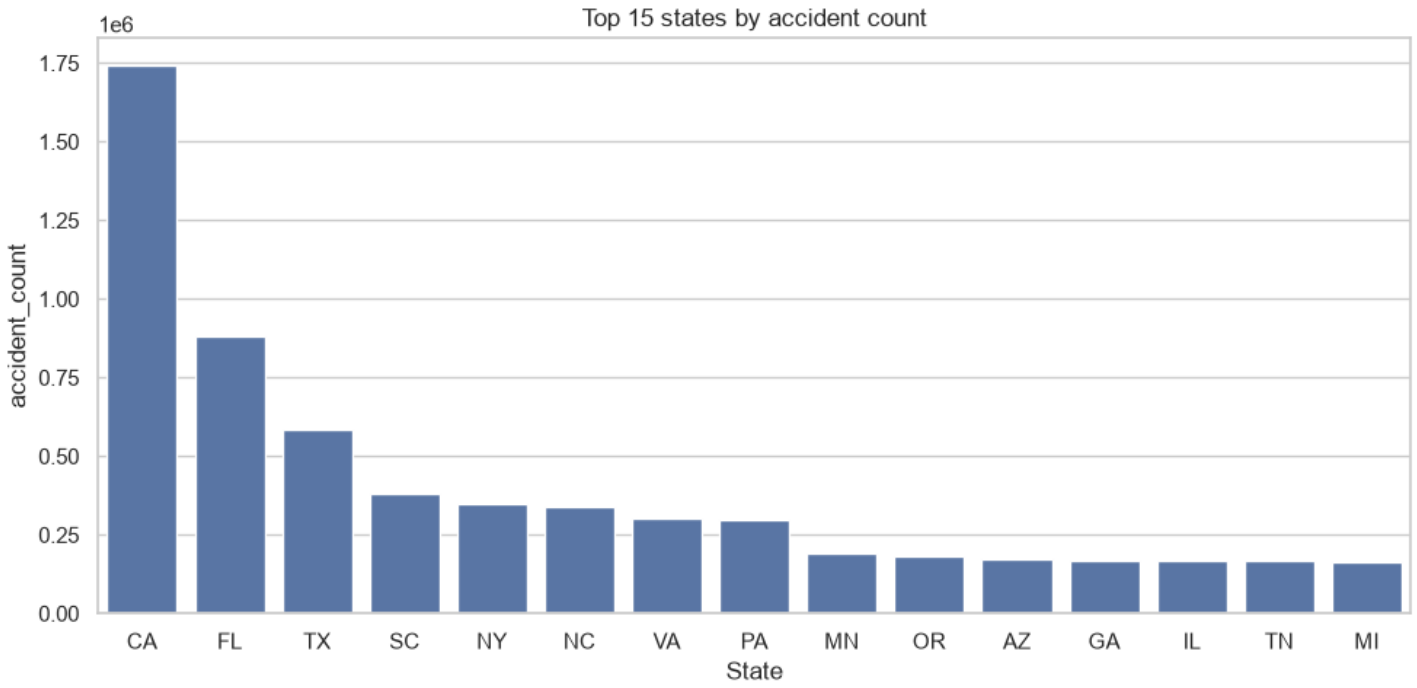
US Accidents (2016-2023): Exploratory Analysis & Hypothesis Testing

Data Analyst/BI portfolio project.

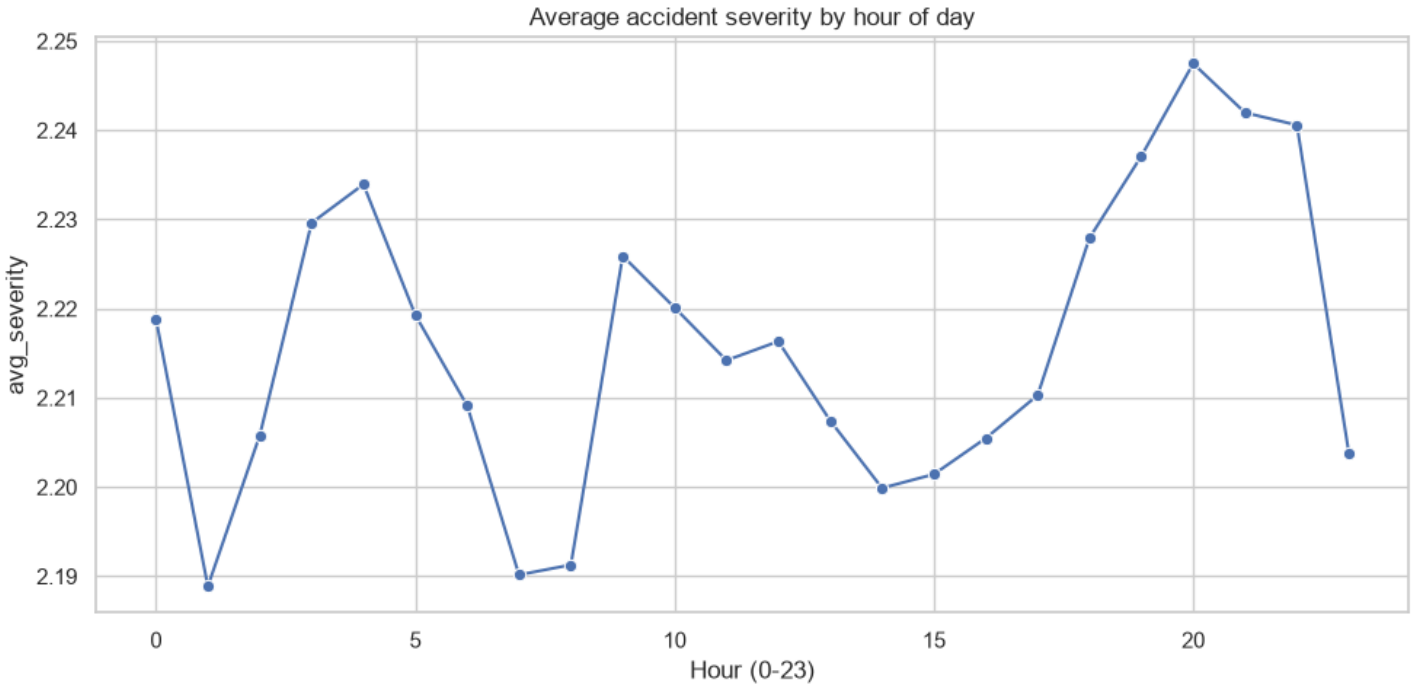
```
"""Setup: load the pre-aggregated Parquet files produced by scripts/build_aggregates.py. This notebook never touches the raw 3GB CSV directly. """ import matplotlib.pyplot as plt import pandas as pd import seaborn as sns from us_accidents.stats_tests import ( anova_severity_by_hour, chi_square_severity_by_weather, logistic_regression_high_severity, ) by_state = pd.read_parquet("../data/processed/by_state.parquet") by_weather = pd.read_parquet("../data/processed/by_weather.parquet") by_hour = pd.read_parquet("../data/processed/by_hour.parquet") sample = pd.read_parquet("../data/processed/sample_for_stats.parquet") sns.set_theme(style="whitegrid")
```

Descriptive EDA

```
fig, ax = plt.subplots(figsize=(10, 5)) top_states = by_state.head(15) sns.barplot(data=top_states, x="State", y="accident_count", ax=ax) ax.set_title("Top 15 states by accident count") plt.tight_layout() plt.savefig("../report/fig_top_states.png", dpi=150) plt.show()
```




```
fig, ax = plt.subplots(figsize=(10, 5)) sns.lineplot(data=by_hour, x="hour", y="avg_severity", marker="o", ax=ax) ax.set_title("Average accident severity by hour of day") ax.set_xlabel("Hour (0-23)") plt.tight_layout() plt.savefig("../report/fig_severity_by_hour.png", dpi=150) plt.show()
```



H1: Severity vs. weather condition

```
"""H1: Accident severity is associated with weather condition.""" chi2_result = chi_square_severity_by_weather(sample) print( f"Chi-square = {chi2_result['statistic']:.2f}, " f"p = {chi2_result['p_value']:.4g}, " f"df = {chi2_result['dof']}" ) chi2_finding = ( f"Severity IS significantly associated with weather condition " f"(chi-square = {chi2_result['statistic']:.2f}, p = {chi2_result['p_value']:.4g})." if chi2_result["significant"] else f"No significant association found between severity and weather " f"condition (chi-square = {chi2_result['statistic']:.2f}, p = {chi2_result['p_value']:.4g})." ) print(chi2_finding)
```

```
Chi-square = 9674.69, p = 0, df = 276 Severity IS significantly associated with weather condition (chi-square = 9674.69, p = 0).
```

(Note: the true p-value underflows to 0 given the sample size and effect size; reported as p < 0.0001 elsewhere in this report rather than literal 0.)

H2: Severity vs. hour of day

```
"""H2: Mean accident severity differs by hour of day.""" anova_result = anova_severity_by_hour(sample) anova_finding = ( f"Mean severity DOES differ significantly by hour of day " f"(F = {anova_result['statistic']:.2f}, p = {anova_result['p_value']:.4g})." if anova_result["significant"] else f"No significant difference in mean severity across hours of day " f"(F = {anova_result['statistic']:.2f}, p = {anova_result['p_value']:.4g})." ) print(anova_finding)
```

```
Mean severity DOES differ significantly by hour of day (F = 9.83, p = 2.797e-35).
```

Predictors of high-severity accidents (logistic regression)

```
"""Logistic regression: which road/weather/time features predict a high-severity accident (Severity >= 3)? """ sample["High_Severity"] = (sample["Severity"] >= 3).astype(int) feature_cols = ["Junction", "Crossing", "Traffic_Signal", "Stop"] sample[feature_cols] = sample[feature_cols].astype(float) logit_result = logistic_regression_high_severity(sample, feature_cols) logit_findings = [] for feature in feature_cols: odds =
```

```
logit_result["odds_ratios"][feature] p = logit_result["p_values"][feature] lower, upper = logit_result["conf_int"][feature] direction = "increases" if odds > 1 else "decreases" sig = "significant" if p < 0.05 else "not significant" line = ( f"{feature}: odds ratio = {odds:.2f} (95% CI [{lower:.2f}, {upper:.2f}]), " f"p = {p:.4g} ({sig}) — presence of {feature} {direction} the odds " f"of a high-severity accident." ) logit_findings.append(line) print(line) pseudo_r2_line = f"Pseudo R-squared: {logit_result['pseudo_r2']:.4f}" print(pseudo_r2_line)
```

```
Junction: odds ratio = 1.34 (95% CI [1.29, 1.39]), p = 9.946e-49 (significant) — presence of Junction increases the odds of a high-severity accident. Crossing: odds ratio = 0.41 (95% CI [0.39, 0.44]), p = 5.143e-208 (significant) — presence of Crossing decreases the odds of a high-severity accident. Traffic_Signal: odds ratio = 0.54 (95% CI [0.52, 0.57]), p = 2.85e-165 (significant) — presence of Traffic_Signal decreases the odds of a high-severity accident. Stop: odds ratio = 0.30 (95% CI [0.27, 0.34]), p = 8.005e-99 (significant) — presence of Stop decreases the odds of a high-severity accident. Pseudo R-squared: 0.0243
```

Findings summary

Based on a reproducible 200,000-row stratified sample (seed 42) of the full 7.7M-row dataset.

H1: Severity vs. weather condition

Severity IS significantly associated with weather condition (chi-square = 9674.69, df = 276, p < 0.0001).

H2: Severity vs. hour of day

Mean severity DOES differ significantly by hour of day (F = 9.83, p = 2.797e-35).

Predictors of high-severity accidents (logistic regression)

Target: `High_Severity` (Severity >= 3). All four road-feature predictors are statistically significant:

- **Junction**: odds ratio = 1.34 (95% CI [1.29, 1.39]), p = 9.946e-49 — presence of a junction *increases* the odds of a high-severity accident.
- **Crossing**: odds ratio = 0.41 (95% CI [0.39, 0.44]), p = 5.143e-208 — presence of a crossing *decreases* the odds of a high-severity accident.
- **Traffic_Signal**: odds ratio = 0.54 (95% CI [0.52, 0.57]), p = 2.85e-165 — presence of a traffic signal *decreases* the odds of a high-severity accident.
- **Stop**: odds ratio = 0.30 (95% CI [0.27, 0.34]), p = 8.005e-99 — presence of a stop sign *decreases* the odds of a high-severity accident.

Pseudo R-squared: 0.0243 — these four road features alone explain a small but statistically robust share of variance in severity; weather and time of day (H1, H2) are additional contributing factors not included in this particular model.

Project decisions

Dataset: US Accidents (2016-2023)

Chosen for scale (~7.7M rows) and popularity, while remaining tractable for a few-day timeline via DuckDB + aggregation rather than loading everything into pandas.

DuckDB for querying, not a hosted database

DuckDB is an embedded analytical engine — it queries the CSV directly off local disk, no server or upload required. A hosted database (e.g. Render's free Postgres) was considered and rejected: free-tier storage caps are too small for the raw file, free databases expire without upgrading to paid, and row-store Postgres is the wrong engine for bulk analytical aggregation compared to DuckDB's columnar execution. The dashboard never needs the raw data anyway — it only reads small pre-aggregated Parquet files.

Manual dataset download instead of the Kaggle API

Simpler for this project's scope — avoids needing Kaggle API credentials or a `.env` file for a one-time download step.

Separate GitHub account (`dsamy-byte`)

Kept fully isolated from the personal GitHub account: git identity is set at the repo level only (no `--global` changes), and a dedicated SSH key with a host alias (`github.com-dsamy-byte`) means this repo authenticates as the new account automatically via its remote URL.

Render over Streamlit Community Cloud

Render hosts the dashboard as a web service, deploying automatically from GitHub on every push to `main`.

Render native Python runtime, not Docker

No Dockerfile — `render.yaml` uses `runtime: python` with a plain build/ start command. Simpler, nothing to maintain, and the dashboard has no dependency (like DuckDB) that actually needs to run at deploy time — it only reads pre-built Parquet files with pandas.

Report/narrative built incrementally

The reports were updated at the end of each project phase (not written only at the end), so there was always a presentable deliverable available, even if the project were paused partway through.

Dashboard and live demo

An interactive Streamlit dashboard covers:

- A **US map** of accident counts by state, plus a top-20 state bar chart.
- A **year-over-year trend** of accident counts, 2016-2023.
- **Cross-filterable** weather-condition and hour-of-day breakdowns — pick a state and/or year and both charts update to that slice of the data.
- A **key drivers panel** surfacing the logistic regression findings (odds ratios with confidence intervals) directly in the dashboard, not just the notebook.

Live dashboard: https://us-accident-analysis-dashboard.onrender.com (Render's free tier sleeps after inactivity — first load may take ~30 seconds to wake.)

Run locally with `uv run streamlit run dashboard/app.py`.

Conclusion

This project demonstrates an end-to-end Data Analyst/BI workflow: scalable data processing (DuckDB) on a real 7.7M-record dataset, rigorous statistical inference (hypothesis testing, logistic regression with interpreted effect sizes) rather than purely descriptive analysis, and a deployed, interactive way to explore the results — with full reproducibility via automated tests and CI/CD.

Project repository: https://github.com/dsamy-byte/us-accident-analysis